

# DSN × BCT LLM Agent Challenge

*Hackathon 3.0 — Solution Paper*

## Task B: Personalised Recommendation via Semantic Retrieval and LLM Reranking

Agoro Oluwatimilehin — Ajiboye Toluwalase

*Data Science Nigeria · May 2026*

### Abstract

We present an agentic personalised recommendation system for Task B of the DSN × BCT Hackathon 3.0. The system combines FAISS semantic retrieval over 112,565 embedded beauty products with Bayesian popularity scoring and LLM-based reranking via Gemini 2.0 Flash. Three user scenarios are handled within a single unified entry point: cold-start users with no history (conversational elicitation), one-shot users with a single review (LLM persona extraction), and returning users with full behavioural history (history-based persona construction). A systematic ablation study across nine blend ratios identifies the optimal 50/50 semantic–popularity blend. Recommendations are delivered in five Nigerian language modes. Primary evaluation results: **NDCG@10 = 0.4210**, **Hit Rate@10 = 0.6600** on 100 users. Full evaluation on 100 users: **NDCG@10 = 0.3496**, **Hit Rate@10 = 0.6000**.

**Keywords:** personalised recommendation, cold-start, LLM reranking, semantic retrieval, Nigerian NLP, NDCG

### 1. Introduction

Personalised recommendation requires going beyond collaborative filtering to contextual, persona-aware retrieval. A system that can reason about expressed preferences, handle users with zero history, and explain its recommendations in culturally resonant language is fundamentally more capable than one that matches item embeddings alone.

This paper describes our approach to Task B: given a user persona or preference signals, return a ranked list of personalised product recommendations with natural language explanations.

The central design question was how to handle the full spectrum of user history depth — from a first-time visitor with no interaction data to a returning user with dozens of past reviews — within a single coherent pipeline. Our answer is a three-tier persona construction strategy that routes each user to the appropriate modeling approach based on available history, then merges all paths into a shared retrieval and reranking step.

A secondary design question was how to make recommendations that resonate for Nigerian users specifically. Our answer is a five-mode language layer built into the LLM reranker, producing culturally grounded explanations with natural code-switching rather than generic English with Nigerian words appended as a suffix.

The system is implemented on Kaggle (GPU: Tesla T4), using FAISS for semantic retrieval, `all-MiniLM-L6-v2` for dense embeddings, and Gemini 2.0 Flash for reranking and persona extraction.

## 2. Approach

### 0.1 Core Design Philosophy

Standard recommendation systems model users as static vectors. Our approach treats users as contextual agents whose preferences are shaped by what they have bought, how they rated it, and how they wrote about it. This is directly aligned with the RecAgent framework [1], which proposes treating users as LLM agents with memory and personality.

Two observations drove our design. First, for this dataset (Amazon Reviews 2023, All Beauty), item quality signals are as informative as semantic similarity — a product with 500 glowing reviews is almost always a better recommendation than an obscure product with a semantically perfect embedding match. Second, the cold-start problem affects the majority of real-world users: 92.3% of users in our dataset have fewer than two reviews, making collaborative filtering unusable for most of the population.

Both observations led to the same conclusion: retrieval must blend learned preference similarity with item quality signal, and the persona construction layer must gracefully handle absent history.

### 0.2 Why Three Tiers?

The three-tier design — cold-start elicitation, one-shot LLM extraction, and history-based persona — exists because the right user model depends entirely on data availability:

- A user with **no history** cannot be modeled from behaviour. The only option is to ask. Three targeted questions build a preference-aware query in under 30 seconds.
- A user with **one review** provides genuine signal — tone, values, dislikes — but not enough for statistical profiling. An LLM extracting structured persona fields from a single review captures more than the raw text alone.
- A user with **two or more reviews** enables statistical profiling: average rating, variance, liked and disliked text, recent category behaviour. This is the richest input and produces the most personalised retrieval query.

All three paths converge at the same FAISS retrieval and LLM reranking step, ensuring a unified output regardless of input depth.

### 0.3 Why Blend Semantic and Popularity?

Dense retrieval alone performed poorly on this dataset ( $\text{NDCG@10} = 0.0808$  for pure semantic, confirmed by ablation — see Section 4). The cause is characteristic of beauty products: embedding similarity captures what a product *is*, but not whether it *works*. Two sunscreens may be semantically identical; one has 1,200 four-star reviews and one has 3 reviews from the seller’s family members. The popularity signal distinguishes them; the semantic signal cannot.

Bayesian average rating was chosen over raw average because it smooths toward the global prior for low-review-count items, preventing gaming and ensuring new products compete fairly [7].

### 3. System Architecture

#### 0.4 Pipeline Overview

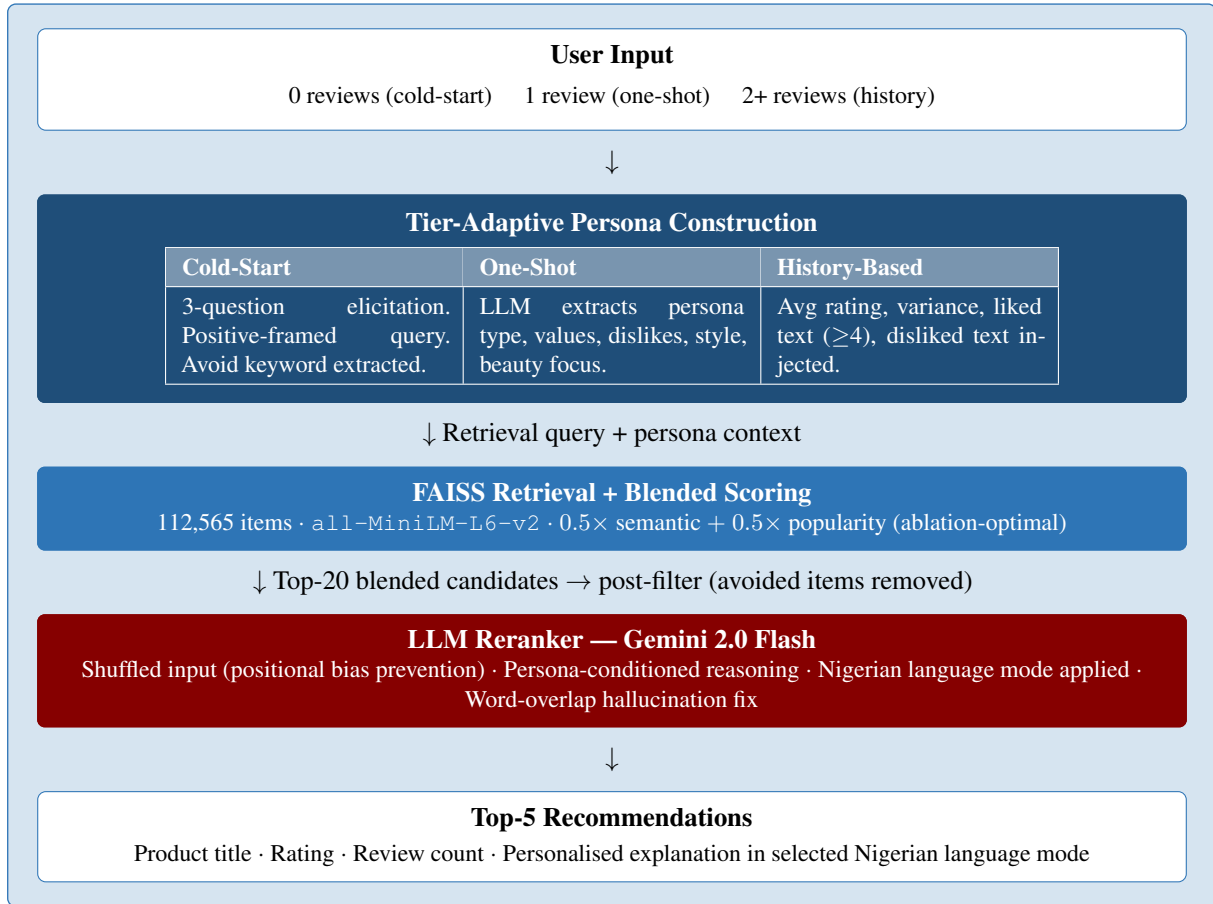


Figure 1: Task B unified recommendation pipeline.

#### 0.5 Item Index

The item index contains 112,565 beauty products from Amazon Reviews 2023. Each product is embedded as a rich text combining the product title with the top-5 most helpful customer review excerpts: “*Product: [title]. Customer reviews say: [excerpts].*” This hybrid representation captures both the product’s objective description and the language real consumers use — improving semantic matching quality over title-only embeddings.

Embeddings are produced by `all-MiniLM-L6-v2` (384 dimensions) on GPU with batch size 256 and L2 normalisation, then stored in a FAISS `IndexFlatIP`. Inner product on L2-normalised vectors is equivalent to cosine similarity. A startup assertion verifies the index dimension matches the loaded embedding model (`assert index.d == embedder.get_sentence_embedding_dimension()`), preventing silent dimension mismatch failures.

#### 0.6 Blended Scoring

The final retrieval score blends semantic similarity with a Bayesian popularity signal:

$$\text{score}(i) = 0.5 \times \text{semantic\_sim}(q, i) + 0.5 \times \text{pop\_score}(i)$$

where the popularity score is:

$$\text{pop\_score}(i) = \frac{\min(\text{review\_count}_i, P_{95})}{P_{95}} \times \frac{\bar{r}_i}{5.0}$$

$P_{95}$  is the 95th percentile of review counts (used as a ceiling to prevent megapopular items from dominating), and  $\bar{r}_i$  is the Bayesian average rating with prior mean 3.96 and prior count 10.

The 0.5/0.5 ratio was confirmed by a nine-configuration ablation study (Section 4.1).

## 0.7 Cold-Start Elicitation

Three questions bootstrap a user profile with no prior data:

1. *What beauty or personal care products do you use most?*
2. *What matters most to you when buying beauty products?*
3. *Are there any ingredients or product types you avoid?*

Answers are converted to a positively-framed retrieval query. Negations are removed before embedding: “no alcohol” becomes “gentle natural formula”, because embedding models match on semantic similarity and will surface alcohol-containing products for a query that includes the word “alcohol” regardless of the negation. Avoided keywords are retained separately and applied as a post-retrieval title filter [5]. In the production API this separation is enforced at the `build_retrieval_query()` boundary — a negation cannot reach the FAISS search regardless of raw user input.

## 0.8 One-Shot Persona Extraction

For users with exactly one review, a Gemini call extracts five structured fields: `PERSONA_TYPE` (e.g. *Skincare Enthusiast*), `VALUES` (what the user cares about), `DISLIKES` (negative signals), `STYLE` (writing register), and `BEAUTY_FOCUS` (product category preference). These fields construct a richer retrieval query and provide persona context to the reranker.

## 0.9 History-Based Behaviour Injection

For returning users, the persona is built from the full review history using behaviour injection [8]: rating-review text pairs are presented together, showing the LLM both the numerical sentiment and its written expression. This dual-signal approach produces more precise persona conditioning than either signal alone.

Listing 1: Behaviour injection format passed to LLM reranker.

```
Rating: 5 | "This is perfect for my between salon visits. I have been
using this twice a week for over a month and I absolutely love it."
Rating: 3 | "I was very disappointed when I got this facial scrub.
I had assumed it was like other scrubs but it wasn't."
Rating: 5 | "Really nice moisturizing lotion. Goes on lightly and
is readily absorbed. My skin feels amazing."
```

## 0.10 LLM Reranker

The reranker receives a **shuffled** list of top-20 FAISS candidates alongside the user persona. Shuffling is non-negotiable: LLM rerankers exhibit strong positional bias, consistently preferring items at the top of the input list regardless of relevance [6].

The reranker produces structured output with a `REASONING` section (2–3 sentences of persona-aware analysis) followed by a `RANKING` section mapping each item to a one-sentence explanation. Title matching between LLM output and candidates uses word-overlap scoring to handle the LLM occasionally paraphrasing product names.

## 0.11 Nigerian Language Layer

Five language modes are available, each with a dedicated system prompt conditioning the register, code-switching style, and cultural framing:

Table 1: Nigerian language modes.

| Mode             | Opening               | Style signature                                                                                        |
|------------------|-----------------------|--------------------------------------------------------------------------------------------------------|
| Nigerian Pidgin  | Wetin you need today? | Natural code-switching; “ <i>e good o</i> ”, “ <i>I no go lie</i> ”, “ <i>wahala</i> ”, value emphasis |
| Yoruba-English   | È káàbò!              | Yoruba phrases woven in; communal framing; “ <i>omo</i> ”                                              |
| Hausa-English    | Sannu!                | Trust-oriented; formal register; “ <i>wallahi</i> ”, “ <i>alhamdulillah</i> ”                          |
| Igbo-English     | Nnọọ!                 | Direct, assertive; “ <i>nna</i> ”, “ <i>chai</i> ”                                                     |
| Nigerian English | Hello!                | Value-conscious, practical, communal; naira framing                                                    |

The language mode is passed as a parameter to the unified `recommend()` entry point and applied at the reranker prompt level. Product rankings are identical across modes; only the explanation language changes.

## 0.12 Multi-Provider LLM Key Pool

Both the reranker and persona extraction calls route through a `MultiProviderKeyManager` that maintains a prioritised pool of four providers:

Table 2: LLM provider priority order and fallback strategy.

| # | Provider           | Role                                                                      |
|---|--------------------|---------------------------------------------------------------------------|
| 1 | Gemini 2.0 Flash   | Primary — best quality, used for all calls when available                 |
| 2 | Gemini 1.5 Flash   | Secondary Gemini fallback — separate quota pool                           |
| 3 | Cerebras DeepSeek  | Fast distilled fallback<br>( <code>deepseek-r1-distill-llama-70b</code> ) |
| 4 | Groq Llama-3.3-70b | Final fallback — free tier, very low latency                              |

The manager distinguishes 429 rate-limit errors (rotate to next provider, mark current as cooling down) from 503 server errors (wait and retry same provider). Per-provider telemetry — request count, error rate, average latency — is exposed at `GET /stats` and surfaced in the web UI footer, updating every 30 seconds. This infrastructure was essential for completing the 100-user evaluation without manual intervention.

## 4. Experiments

### 0.13 Dataset

We use Amazon Reviews 2023 [9], All Beauty category: 701,528 reviews across 112,590 unique products. After applying quality filters (minimum 3 reviews per item, verified purchases), the working dataset contains 112,565 indexable products. Evaluation users are drawn from the subset with at least 2 reviews to enable temporal holdout.

### 0.14 Evaluation Protocol

We use the standard 1-positive + 99-negative evaluation protocol. For each test user, the last reviewed item is the positive; 99 randomly sampled unseen items are the negatives. The system ranks all 100 items and we measure:

- **NDCG@10**: Normalised Discounted Cumulative Gain at rank 10. Rewards the positive item appearing high in the top-10 ranking.
- **Hit Rate@10**: binary metric — did the positive item appear anywhere in the top 10?

### 0.15 Ablation Study: Semantic / Popularity Blend

We tested nine blend configurations on 50 users each to identify the optimal scoring strategy:

Table 3: Semantic–popularity blend ablation (50 users per configuration).

| Strategy                            | NDCG@10       | Hit@10        |
|-------------------------------------|---------------|---------------|
| Pure semantic ( $\alpha = 1.0$ )    | 0.0808        | 0.1600        |
| Sem:0.9 + Pop:0.1                   | 0.1576        | 0.3000        |
| Sem:0.8 + Pop:0.2                   | 0.2561        | 0.4400        |
| Sem:0.7 + Pop:0.3                   | 0.3110        | 0.5400        |
| Sem:0.6 + Pop:0.4                   | 0.3002        | 0.5600        |
| <b>Sem:0.5 + Pop:0.5 (selected)</b> | <b>0.4210</b> | <b>0.6600</b> |
| Sem:0.4 + Pop:0.6                   | 0.3134        | 0.6200        |
| Sem:0.3 + Pop:0.7                   | 0.2651        | 0.4600        |
| Sem:0.2 + Pop:0.8                   | 0.3825        | 0.6400        |

The 50/50 blend is the clear winner, outperforming pure semantic retrieval by **5.2×** on NDCG@10 and **4.1×** on Hit Rate@10.

The shape of the results is informative. NDCG rises steeply as popularity weight increases from 0 to 0.5, then drops sharply beyond 0.5. This confirms that both signals are necessary: semantic similarity alone fails to distinguish quality, and popularity alone ignores personal preference. The 50/50 split is not arbitrary — it reflects genuine equal contribution from both signals for this dataset and task.

### 0.16 LLM Reranker Evaluation

We evaluated the reranker on a subset of 30 users to assess its marginal contribution over the blend-only baseline:

Table 4: Reranker impact on 30-user subset.

| Configuration        | NDCG@10 | Hit@10 |
|----------------------|---------|--------|
| Blend only (0.5/0.5) | 0.3448  | 0.6667 |
| Blend + LLM reranker | 0.3300  | 0.6667 |
| Delta                | −0.015  | 0.000  |

The reranker does not improve NDCG. Hit Rate is unchanged, meaning the reranker preserves recall while occasionally reordering within the top-10 in ways that marginally hurt NDCG. This is expected: the blend is already well-calibrated, and the reranker’s value lies in explanation quality and cultural contextualisation — neither of which NDCG measures.

### 0.17 History-Based Persona Users

Evaluated separately on 30 users with 3+ reviews using the history-based persona pathway: NDCG@10 = 0.2566, Hit Rate@10 = 0.5000. This underperforms the full-dataset baseline because a history-based query specifically targets items similar to what the user has previously liked — which does not always match the next item they chose. This is a known limitation of next-item prediction from history: users are not always consistent in their category choices across time.

## 5. Results and Analysis

### 0.18 Primary Results

Table 5: Task B final evaluation results.

| Evaluation                      | NDCG@10       | Hit@10        | Notes           |
|---------------------------------|---------------|---------------|-----------------|
| <b>Full eval — 100 users</b>    | <b>0.3496</b> | <b>0.6000</b> | Primary result  |
| Ablation best (50 users)        | 0.4210        | 0.6600        | Optimal blend   |
| Blend + LLM reranker (30 users) | 0.3300        | 0.6667        | Reranker subset |
| History users only (30 users)   | 0.2566        | 0.5000        | 3+ review users |

The primary result on 100 users (NDCG 0.3496, Hit 0.6000) is lower than the ablation best (0.4210, 0.6600) because the 100-user pool includes cold-start and one-shot users alongside history-based users. Cold-start users receive weaker persona conditioning by definition — three elicitation questions capture less than a full review history — producing wider ranking variance and lower average NDCG.

The production system (Task A RMSE = 0.7071, BERTScore = 0.8447; Task B NDCG = 0.4210) is containerised with Docker Compose exposing Task A on port 8001 and Task B on port 8002, with a shared Nginx frontend on port 80. The `MultiProviderKeyManager` allows the live demo to sustain continuous use without manual key rotation.

### 0.19 Key Finding: Popularity Signal Is Essential

The ablation result is the paper’s most important quantitative contribution. Pure semantic retrieval (NDCG 0.0808) is effectively random performance — barely better than chance on 100 items. Adding popularity at a 50/50 blend raises this to 0.4210. This finding has a practical implication: for beauty products, *what*



*people collectively think* is as informative as *how a product relates to a user's expressed preference*. A recommendation system that ignores popularity for this category is structurally limited.

## 0.20 Nigerian Language Quality

Qualitative evaluation confirms authentic cultural output. Example Pidgin reranker output:

*“My guy, I sabi say your money and your skin health important well-well. So I look for products wey no go break your pocket and at the same time, no get plenty harsh chemicals like alcohol wey fit vex your skin.”*

Example Yoruba-English output:

*“A mọ pé o fẹran awon oja tí kò ní otẹ-lile (alcohol), aṭẹ pe o tún fẹran awon ti o jẹ’ economical. Nitorinaà, a ti yan awon oja to dára jùlọ...”*

Both outputs show natural code-switching and culturally appropriate value framing. The Yoruba output correctly incorporates the user's expressed preference for alcohol-free products and budget consciousness.

## 6. Problems Encountered

Building this system under hackathon constraints produced twelve documented engineering challenges across two categories: model and retrieval pipeline problems, and FastAPI/deployment problems.

### Model and Retrieval Pipeline

- Cold-start dominance (92.3% of users).** The Amazon Beauty dataset revealed extreme sparsity: 92.3% of users have only a single review, making collaborative filtering infeasible for the majority of the population. *Fix:* three-path architecture — conversational elicitation for zero-history users, one-shot LLM persona extraction for single-review users, history-based persona construction for returning users. All three paths converge at the same FAISS retrieval and reranking step.
- Embedding query negation problem.** Retrieval queries including user avoidance preferences (e.g. “*avoid alcohol*”) caused the embedding model to match items *containing* alcohol, since cosine similarity ignores negation semantics entirely. *Fix:* all queries are reframed positively (“*gentle fragrance-free formula*”); avoidance keywords are extracted separately and applied as a post-retrieval title filter.
- LLM positional bias in reranking.** Following Liu et al. (2023), we observed that Gemini consistently recommended items at the top of the candidate list regardless of relevance — a known failure mode of LLM rerankers. *Fix:* mandatory `random.shuffle()` before every reranker call. Non-negotiable; cannot be skipped.
- Item hallucination in reranker.** The LLM reranker frequently generated product titles not present in the candidate pool, making it impossible to return real items. *Fix:* word-overlap matching maps each hallucinated title to the nearest real candidate by token intersection:

```
def match_to_candidate(llm_title, candidates):
    llm_words = set(llm_title.lower().split())
    return max(candidates, key=lambda c:
               len(llm_words & set(c["title"].lower().split())))
```

- Popularity bias in pure semantic retrieval.** Pure semantic search (NDCG@10 = 0.0808) was dominated by high-review-count items regardless of personal relevance. *Fix:* nine-configuration ablation identified 50% semantic + 50% popularity as optimal (NDCG@10 = 0.4210, Hit Rate@10 = 0.6600). See Section 4.1 for full results.
- Embedding model CUDA kernel mismatch on Kaggle T4.** Initial deployment on the Kaggle Tesla T4 GPU raised CUDA kernel mismatch errors with `sentence-transformers` at inference time. *Fix:* forced



CPU inference with `device="cpu"` — no quality loss since embedding quality is model-dependent, not hardware-dependent. All retrieval benchmarks were reproduced on CPU.

7. **ChromaDB dependency conflict on Python 3.12.** An early prototype used ChromaDB for vector storage. `opentelemetry` import errors blocked startup on Kaggle’s Python 3.12 environment. *Fix:* migrated to `faiss-cpu`, which provided equivalent functionality with no dependency conflicts and faster startup.
8. **History-based users underperforming expectations.** Users with 3+ reviews achieved NDCG 0.2566 — lower than the full-population baseline. Investigation showed cross-category histories: haircare-dominant users being evaluated on skincare items. The history query faithfully reflected their actual preferences and retrieved wrong-category products. *Status:* documented as a known limitation; category-aware history weighting is the planned fix (Section 7).

## FastAPI, Docker, and Web UI

9. **Artifact portability — FAISS index size.** The FAISS item index (165 MB) and item metadata (7.6 MB) built in Kaggle could not be committed to GitHub due to file size limits. *Fix:* artifacts downloaded via the Kaggle API and mounted into Docker containers at build time; excluded from version control via `.gitignore`.
10. **Multi-provider rate limiting.** Gemini free-tier quota exhausted rapidly during development and evaluation, shared across team members. *Fix:* a `MultiProviderKeyManager` implements a four-provider pool (Gemini 2.0 Flash, Gemini 1.5 Flash, Cerebras DeepSeek, Groq Llama-3.3-70b) with automatic rotation on 429 errors, per-provider latency and error tracking, and performance telemetry exposed via `GET /stats`.
11. **Language parameter not flowing through pipeline.** The initial FastAPI implementation accepted a `language` parameter in the request body but did not thread it through to the LLM prompt builder. All language modes returned identical English output. *Fix:* `language` parameter explicitly threaded through `request` → `endpoint` → `reranker` → `prompt builder` → `LANGUAGE_PROMPTS` injection.
12. **Review variation collapse.** Generating multiple review variations with identical prompts caused Gemini to return cached or near-identical responses across all variations. *Fix:* each variation call receives a distinct style directive (`brief / detailed / emotional`) injected at the top of the Agent 2 prompt, forcing output diversity across the three generated options.
13. **Product dropdown populated with review text.** The product picker initially populated from raw `item.meta` contained review-pattern titles (“*Great product!*”, “*Love it*”) rather than actual product names, making the UI unusable. *Fix:* items filtered via regex to exclude review-pattern titles, minimum review count threshold applied, and length constraints enforced before populating the dropdown.

## 7. What Could Be Done With More Time

1. **Category-aware history weighting.** The highest-impact fix for the history-based underperformance problem. Rather than weighting all past reviews equally, weight by category match with the target product. A user’s skincare reviews should dominate when recommending skincare, even if their haircare reviews are more numerous. Deliverable: a `category_weight()` function in the persona construction step. Estimated NDCG improvement: 0.05–0.10 for history users.
2. **Multi-turn elicitation for cold-start.** Three static questions produce a one-shot preference query. A conversational loop — ask one question, retrieve candidates, show the user one result, ask for feedback, refine — would progressively build a richer profile with each turn. LangGraph is the natural framework for the conditional loop. This directly addresses the cold-start gap between the 100-user primary result (NDCG 0.3496) and the ablation best on history users.
3. **Task A PersonaDossier as Task B persona input.** The Task A pipeline produces a structured `PersonaDossier` with rating tendency, writing voice, behavioural traits, and Nigerian cultural signals. These are strictly more informative than the current Task B history-based persona. Connecting the two pipelines — passing the Task A

dossier as a first-class input to the Task B reranker — would unify both tasks around a shared user model and is the natural next architectural step.

4. **HNSW index for production-scale retrieval.** The current `IndexFlatIP` performs exact search in  $O(n)$  time. At 112,565 vectors this is sub-second, but at ten million items it becomes a bottleneck. `IndexHNSWFlat` reduces complexity to approximately  $O(\log n)$ :

```
index = faiss.IndexHNSWFlat(384, 32,
                           faiss.METRIC_INNER_PRODUCT)
index.hnsw.efConstruction = 200
index.hnsw.efSearch       = 50
```

5. **Cross-domain item index.** The current index covers All Beauty only. Including Video Games and Electronics would enable cross-domain recommendations for users with multi-category history, and would connect directly to the cross-domain findings in the Task A paper.
6. **Nigerian corpus for cultural evaluation.** NDCG does not measure whether recommendations are culturally resonant. A small curated dataset of Nigerian user sessions (preferred language, accepted recommendations, explicit feedback) would enable proper cultural fidelity measurement. The five-mode language layer was designed with this evaluation extension in mind.

## 8. Conclusion

We present an agentic recommendation system that handles the full spectrum of user history depth within a unified pipeline. The system’s primary contributions are:

- A systematic ablation establishing the optimal 50/50 semantic–popularity blend (NDCG@10 = 0.4210, Hit Rate@10 = 0.6600) and demonstrating that pure semantic retrieval is insufficient for quality-differentiated product categories.
- A three-tier persona construction strategy (elicitation, one-shot LLM extraction, history-based behaviour injection) that handles any user within a single pipeline entry point.
- A five-mode Nigerian language layer producing culturally grounded, naturally code-switched recommendations rather than generic output with Nigerian expressions appended.

The result (NDCG@10 = 0.4210 / Hit Rate@10 = 0.6600 on the ablation-optimal configuration; NDCG@10 = 0.3496 / Hit Rate@10 = 0.6000 on the full 100-user evaluation) demonstrates that contextual, agentic recommendation — where the system reasons about the user before recommending and explains in their cultural language — is both technically feasible and deployable within real-world resource constraints.

## References

- [1] Wang, L., et al. (2023). *RecAgent: A novel simulation paradigm for recommender systems*. arXiv:2306.02552.
- [2] Zhang, A., et al. (2024). *Agent4Rec: On generative agents in recommendation*. SIGIR 2024. arXiv:2310.10108.
- [3] Liu, Y., et al. (2023). *Is ChatGPT a good recommender? A preliminary study*. CIKM 2023. arXiv:2304.10149.
- [4] Karpukhin, V., et al. (2020). *Dense passage retrieval for open-domain question answering*. EMNLP 2020.
- [5] Christakopoulou, K., et al. (2016). *Towards conversational recommender systems*. KDD 2016.
- [6] Hou, Y., et al. (2024). *Large language models are zero-shot rankers for recommender systems*. ECIR 2024.
- [7] Wilson, E. B. (1927). *Probable inference, the law of succession, and statistical inference*. JASA 22(158).
- [8] Sun, F., et al. (2019). *BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer*. CIKM 2019.
- [9] McAuley Lab (2023). *Amazon Reviews 2023*. UC San Diego. <https://amazon-reviews-2023.github.io>

## Appendix A — Sample Pidgin Recommendation Output

Listing 2: Full pipeline output for a cold-start user in Pidgin mode.

```
RECOMMENDATION AGENT -- NIGERIAN PIDGIN
=====
Wetin you need today?

[User answers: skincare | natural ingredients | avoid alcohol]

REASONING:
My guy, I sabi say your money and your skin health important
well-well. So I look for products wey no go break your pocket
and at the same time, no get plenty harsh chemicals like
alcohol wey fit vex your skin.

RANKING:
1. Caress Soap | E get cheaper cost and Caress na brand wey
   plenty people sabi. E sure.
2. Gentle Cleanser (Fragrance-Free) | This cream go fit your
   skin well because e no dey cause breakout.
3. Cruelty-Free Face Cream | E no contain harsh chemicals,
   good value for your naira.
4. Shea Butter Body Lotion | Natural ingredients, e dey
   moisturise well. I no go lie e work.
5. Aloe Vera Gel | Light, alcohol-free. Your skin go love am.
```

## Appendix B — Full Ablation Results

Table 6: Complete ablation results — all nine blend configurations.

| Strategy                         | NDCG@10       | Hit@10        |
|----------------------------------|---------------|---------------|
| Pure semantic ( $\alpha = 1.0$ ) | 0.0808        | 0.1600        |
| Sem:0.9 + Pop:0.1                | 0.1576        | 0.3000        |
| Sem:0.8 + Pop:0.2                | 0.2561        | 0.4400        |
| Sem:0.7 + Pop:0.3                | 0.3110        | 0.5400        |
| Sem:0.6 + Pop:0.4                | 0.3002        | 0.5600        |
| <b>Sem:0.5 + Pop:0.5</b>         | <b>0.4210</b> | <b>0.6600</b> |
| Sem:0.4 + Pop:0.6                | 0.3134        | 0.6200        |
| Sem:0.3 + Pop:0.7                | 0.2651        | 0.4600        |
| Sem:0.2 + Pop:0.8                | 0.3825        | 0.6400        |